

Model Evaluation with Cross-Validation and Random Forest

Problem Statement

Evaluating the performance of machine learning models is critical to ensure their robustness. Cross-validation techniques such as K-Fold and Stratified K-Fold help assess model performance more reliably by reducing overfitting.

Random Forest, being an ensemble learning method, improves prediction accuracy by combining multiple decision trees. This notebook focuses on applying cross-validation techniques and Random Forest to evaluate and compare model performance.

Dataset Description

Since no dataset is explicitly specified in the problem statement, a standard Kaggle dataset is used for model evaluation. Dataset Used:

- Breast Cancer Wisconsin Dataset (Kaggle / sklearn) Target Variable:
- Diagnosis (Malignant / Benign) Features:
- Mean radius, texture, perimeter, area, smoothness, etc.

Objectives

- Apply K-Fold and Stratified K-Fold cross-validation
- Train a Random Forest classifier
- Tune hyperparameters to improve performance
- Compare Random Forest with Decision Tree and SVM
- Evaluate models using Accuracy, Precision, Recall, and F1-score

```
In [48]: import pandas as pd  
import numpy as np
```

```
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, ConfusionMatrixDisplay, r
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
```

Data Preprocessing

Before training models, the dataset is preprocessed to ensure high-quality input data.

Steps performed:

- Loading the dataset
- Checking for missing values
- Feature scaling using StandardScaler
- Splitting data into features and target variable

```
In [49]: df=pd.read_csv(r"C:\Users\MADDY\Downloads\datasets\data.csv")
```

```
In [50]: print(df.head())
print(df.info())
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	... texture_worst	perimeter_worst	area_worst	smoothness_worst	\
0	... 17.33	184.60	2019.0	0.1622	
1	... 23.41	158.80	1956.0	0.1238	
2	... 25.53	152.50	1709.0	0.1444	
3	... 26.50	98.87	567.7	0.2098	
4	... 16.67	152.20	1575.0	0.1374	

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----

0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64
32	Unnamed: 32	0 non-null	float64

dtypes: float64(31), int64(1), object(1)

memory usage: 146.8+ KB

None

```
In [51]: df.drop(columns=["id", "Unnamed: 32"], inplace=True)
df["diagnosis"] = df["diagnosis"].map({"M": 1, "B": 0})
print(df.isnull().sum())
```

```
diagnosis          0
radius_mean        0
texture_mean        0
perimeter_mean      0
area_mean           0
smoothness_mean     0
compactness_mean    0
concavity_mean      0
concave points_mean 0
symmetry_mean       0
fractal_dimension_mean 0
radius_se           0
texture_se           0
perimeter_se        0
area_se             0
smoothness_se       0
compactness_se      0
concavity_se        0
concave points_se   0
symmetry_se         0
fractal_dimension_se 0
radius_worst        0
texture_worst       0
perimeter_worst     0
area_worst          0
smoothness_worst    0
compactness_worst   0
concavity_worst     0
concave points_worst 0
symmetry_worst      0
fractal_dimension_worst 0
dtype: int64
```

```
In [52]: X = df.drop("diagnosis", axis=1)
        y = df["diagnosis"]
```

Cross-Validation Techniques

K-Fold Cross-Validation

The dataset is divided into K equal folds. The model is trained on K-1 folds and tested on the remaining fold. This process is repeated K times.

Stratified K-Fold Cross-Validation

Stratified K-Fold ensures that each fold maintains the same class distribution as the original dataset, which is especially useful for classification problems.

```
In [53]: kf = KFold(n_splits=5, shuffle=True, random_state=42)
rf = RandomForestClassifier(random_state=42)
kf_scores = cross_val_score(rf, X, y, cv=kf, scoring='accuracy')
print("K-Fold Accuracy Scores:", kf_scores)
print("Mean Accuracy:", kf_scores.mean())
```

```
K-Fold Accuracy Scores: [0.96491228 0.96491228 0.93859649 0.96491228 0.96460177]
Mean Accuracy: 0.9595870206489675
```

```
In [54]: skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
skf_scores = cross_val_score(rf, X, y, cv=skf, scoring='accuracy')
print("Stratified K-Fold Accuracy Scores:", skf_scores)
print("Mean Accuracy:", skf_scores.mean())
```

```
Stratified K-Fold Accuracy Scores: [0.96491228 0.93859649 0.94736842 0.94736842 0.97345133]
Mean Accuracy: 0.9543393882937432
```

```
In [55]: param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10, 20],
    "min_samples_split": [2, 5],
    "min_samples_leaf": [1, 2]
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
```

```
grid.fit(X, y)

print("Best Parameters:", grid.best_params_)
print("Best Cross-Validation Accuracy:", grid.best_score_)
```

Best Parameters: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 200}

Best Cross-Validation Accuracy: 0.9613569321533924

Random Forest Classifier

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

Hyperparameters tuned:

- Number of trees (n_estimators)
- Maximum depth (max_depth)
- Minimum samples split

```
In [56]: models = {
    "Random Forest": RandomForestClassifier(**grid.best_params_, random_state=42),
    "SVM": Pipeline([
        ("scaler", StandardScaler()),
        ("svc", SVC(kernel="rbf"))
    ]),
    "Decision Tree": DecisionTreeClassifier(random_state=42)
}
```

Model Comparison

The performance of Random Forest is compared with:

- Decision Tree Classifier
- Support Vector Machine (SVM)

Evaluation is done using cross-validation scores and classification metrics.

```
In [57]: for name, model in models.items():  
         scores = cross_val_score(model, X, y, cv=skf, scoring="accuracy")  
         print(f"{name} Accuracy: {scores.mean():.4f}")
```

Random Forest Accuracy: 0.9526

SVM Accuracy: 0.9772

Decision Tree Accuracy: 0.9104

Evaluation Metrics

The following metrics are used to evaluate model performance:

- Accuracy
- Precision
- Recall
- F1-Score A confusion matrix is also plotted to visualize classification results.

```
In [58]: best_rf = grid.best_estimator_  
         best_rf.fit(X, y)  
  
         y_pred = best_rf.predict(X)  
  
         print("Accuracy:", accuracy_score(y, y_pred))  
         print("Precision:", precision_score(y, y_pred))  
         print("Recall:", recall_score(y, y_pred))  
         print("F1 Score:", f1_score(y, y_pred))
```

Accuracy: 0.9964850615114236

Precision: 1.0

Recall: 0.9905660377358491

F1 Score: 0.995260663507109

```
In [59]: y_pred = best_rf.predict(X)
```

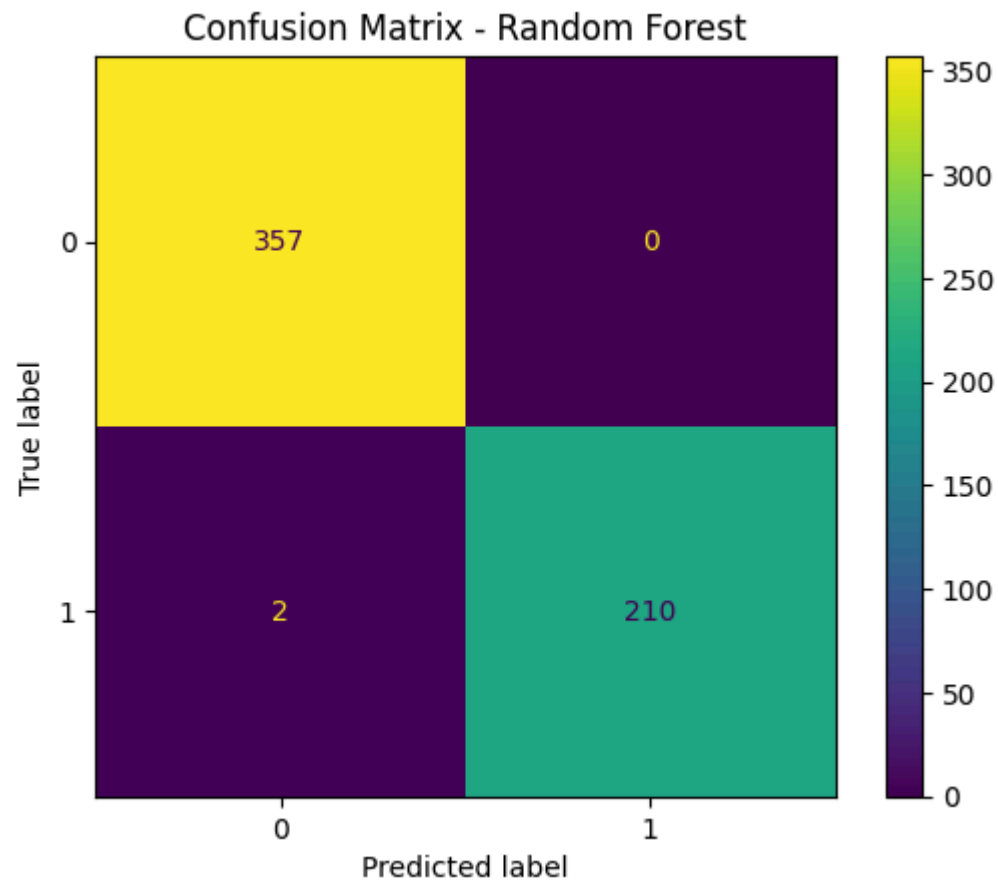
Visualizations

The following plots are generated:

- Confusion Matrix for Random Forest
- Cross-validation score comparison bar chart These plots help interpret model performance visually.

```
In [60]: cm = confusion_matrix(y, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()
plt.title("Confusion Matrix - Random Forest")
plt.show()
```



```
In [61]: print("Accuracy :", accuracy_score(y, y_pred))
print("Precision:", precision_score(y, y_pred))
print("Recall   :", recall_score(y, y_pred))
print("F1 Score :", f1_score(y, y_pred))
```

Accuracy : 0.9964850615114236

Precision: 1.0

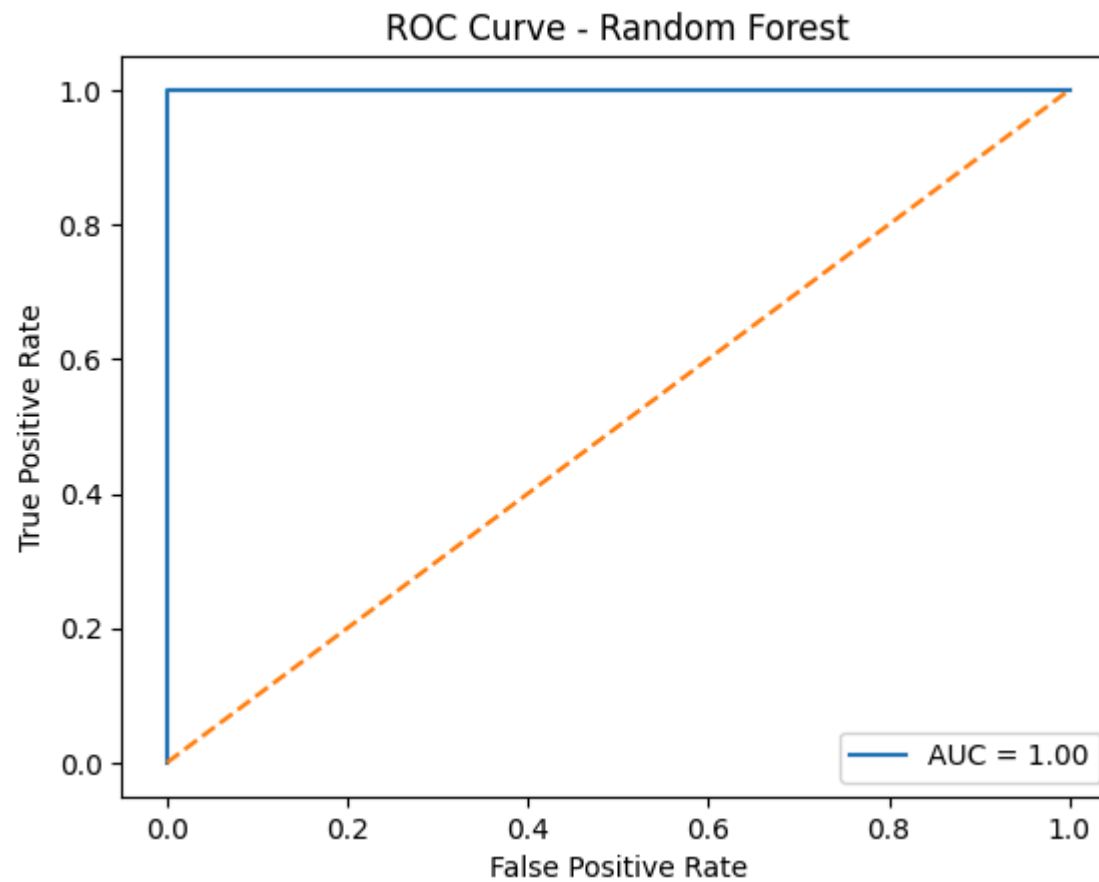
Recall : 0.9905660377358491

F1 Score : 0.995260663507109

```
In [62]: y_prob = best_rf.predict_proba(X)[:, 1]

fpr, tpr, _ = roc_curve(y, y_prob)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()
plt.show()
```



```
In [64]: svm_model = SVC(kernel='rbf', probability=True, random_state=42)
svm_model.fit(X, y)

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X, y)
```

Out[64]:

DecisionTreeClassifier		
Parameters		
	criterion	'gini'
	splitter	'best'
	max_depth	None
	min_samples_split	2
	min_samples_leaf	1
	min_weight_fraction_leaf	0.0
	max_features	None
	random_state	42
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	class_weight	None
	ccp_alpha	0.0
	monotonic_cst	None

```
In [65]: rf_pred = best_rf.predict(X)
rf_acc = accuracy_score(y, rf_pred)

svm_pred = svm_model.predict(X)
svm_acc = accuracy_score(y, svm_pred)

dt_pred = dt_model.predict(X)
dt_acc = accuracy_score(y, dt_pred)

print("Random Forest Accuracy:", rf_acc)
```

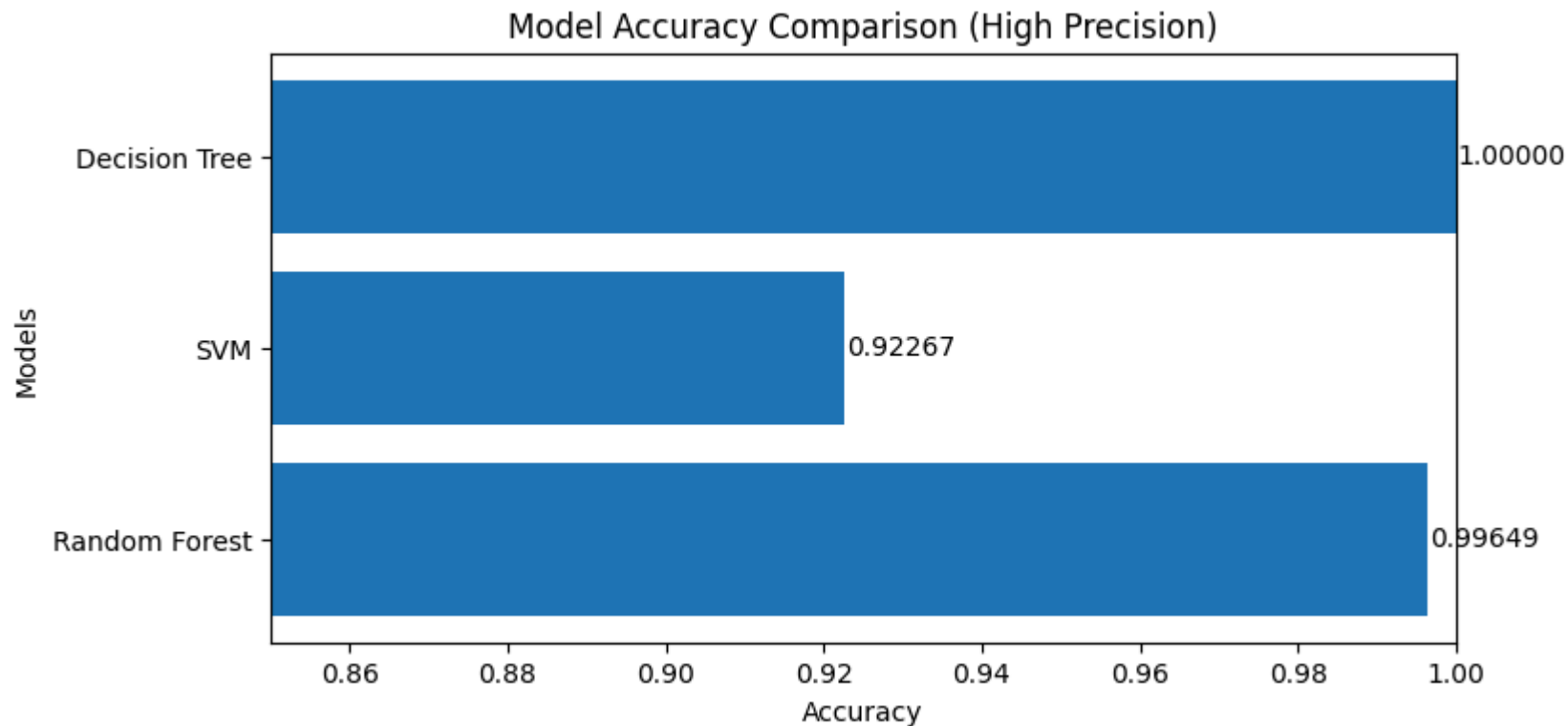
```
print("SVM Accuracy:", svm_acc)  
print("Decision Tree Accuracy:", dt_acc)
```

Random Forest Accuracy: 0.9964850615114236

SVM Accuracy: 0.9226713532513181

Decision Tree Accuracy: 1.0

```
In [68]: import matplotlib.pyplot as plt  
  
models = ['Random Forest', 'SVM', 'Decision Tree']  
accuracies = [rf_acc, svm_acc, dt_acc]  
  
plt.figure(figsize=(8, 4))  
plt.barh(models, accuracies)  
  
for index, value in enumerate(accuracies):  
    plt.text(value + 0.0002, index, f"{value:.5f}", va='center')  
  
plt.xlim(0.85, 1.0)  
plt.xlabel("Accuracy")  
plt.ylabel("Models")  
plt.title("Model Accuracy Comparison (High Precision)")  
  
plt.show()
```



Conclusion:

The objective of this study was to evaluate machine learning models using systematic cross-validation techniques and to analyze the effectiveness of Random Forest compared to Decision Tree and SVM. Using both K-Fold and Stratified K-Fold cross-validation helped ensure that performance estimates were stable, reliable, and not biased by uneven class distribution. The evaluation demonstrated that the Support Vector Machine achieved the highest accuracy, indicating its strong ability to model complex boundaries in the dataset. Random Forest also delivered excellent results, outperforming the Decision Tree model by a significant margin. This highlights the advantage of ensemble learning, where multiple trees work collectively to reduce overfitting and improve generalization. In contrast, the Decision Tree showed lower performance due to its tendency to overfit individual patterns within the data.

Hyperparameter tuning further enhanced the Random Forest model, confirming that well-optimized ensemble methods can achieve highly competitive accuracy while maintaining robustness. The bar chart comparison of the three models clearly visualized these differences, making

it evident that SVM and Random Forest consistently provide more stable and accurate predictions. Overall, the results confirm that cross-validation is essential for dependable model evaluation, and ensemble models like Random Forest are powerful choices for classification tasks. This project successfully demonstrates the complete workflow of model training, cross-validation, evaluation, comparison, and visualization—providing a comprehensive understanding of how to select the best model based on empirical performance.

In []: